Kubernetes

HPAでPodの自動スケーリングを実現する

CPU/メモリベースの自動スケーリング設計

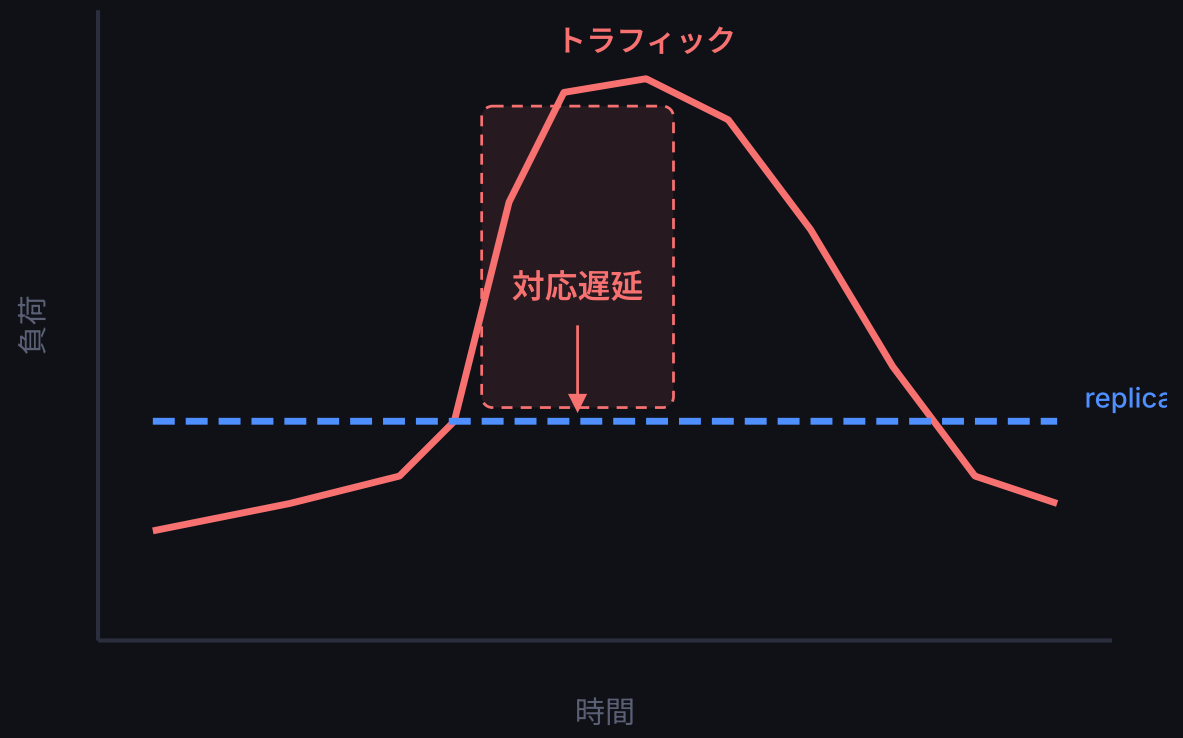
viserhaut — 2026

SECTION 01

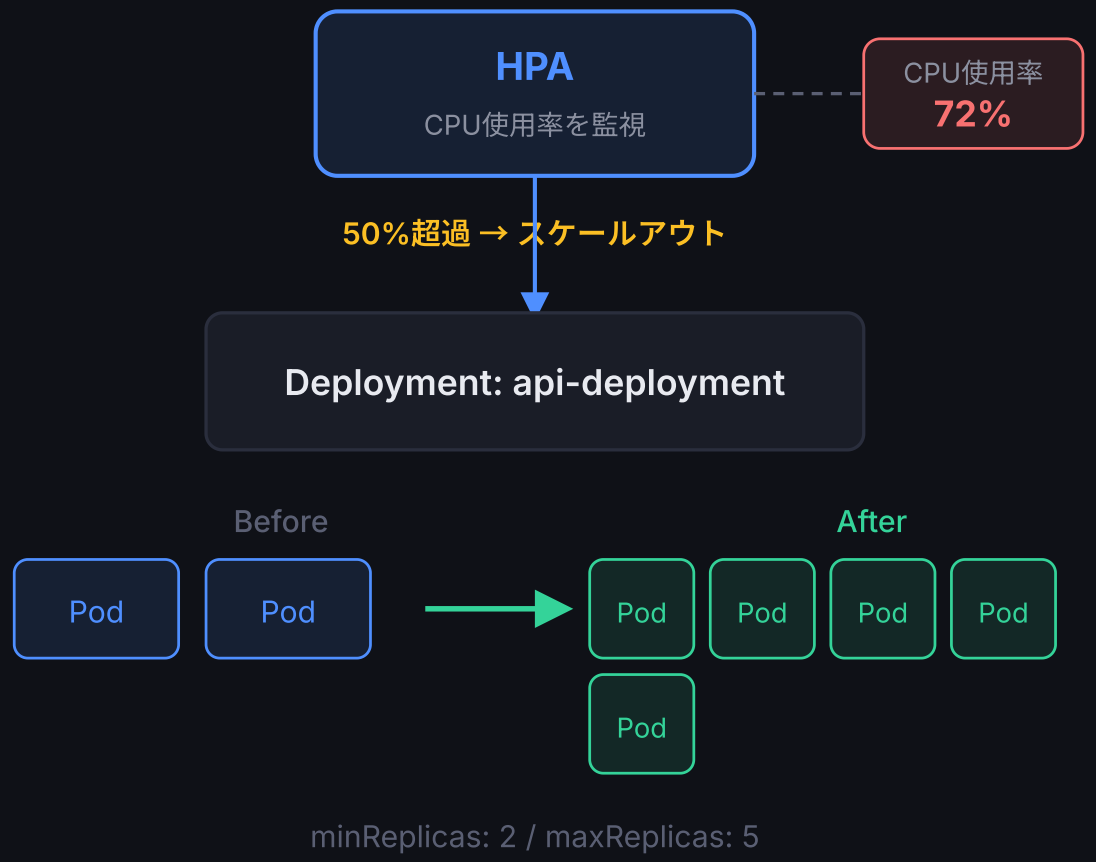
なぜ自動スケーリングが必要か

手動スケーリングでは負荷急増に対応できない

- アクセス急増時にreplicas手動変更では**数分のダウンタイム**が発生する
- 負荷が下がった後のスケールイン忘れで**不要なコスト**が発生し続ける
- 深夜・休日の負荷変動に**人手で対応するのは非現実的**



HPAがメトリクスを監視してPod数を自動調整する



- HPAは**requestsに対するCPU使用率**を基準にスケーリングを判断する
- 閾値を超えるとPodを**自動でスケールアウト**し、下がれば**スケールイン**する
- minReplicas / maxReplicasで**スケール範囲を制御**し、暴走を防ぐ

ECSオートスケーリング vs HPA — 設定構造の比較

観点	ECS	HPA
スケール対象	Service（タスク数）	Deployment（Pod数）
メトリクス基準	CloudWatch Alarm	Metrics Server（requests基準）
設定方法	ターゲット + ポリシーの2リソース	HPAマニフェスト1つで完結

HPA マニフェストとスケーリングポリシー

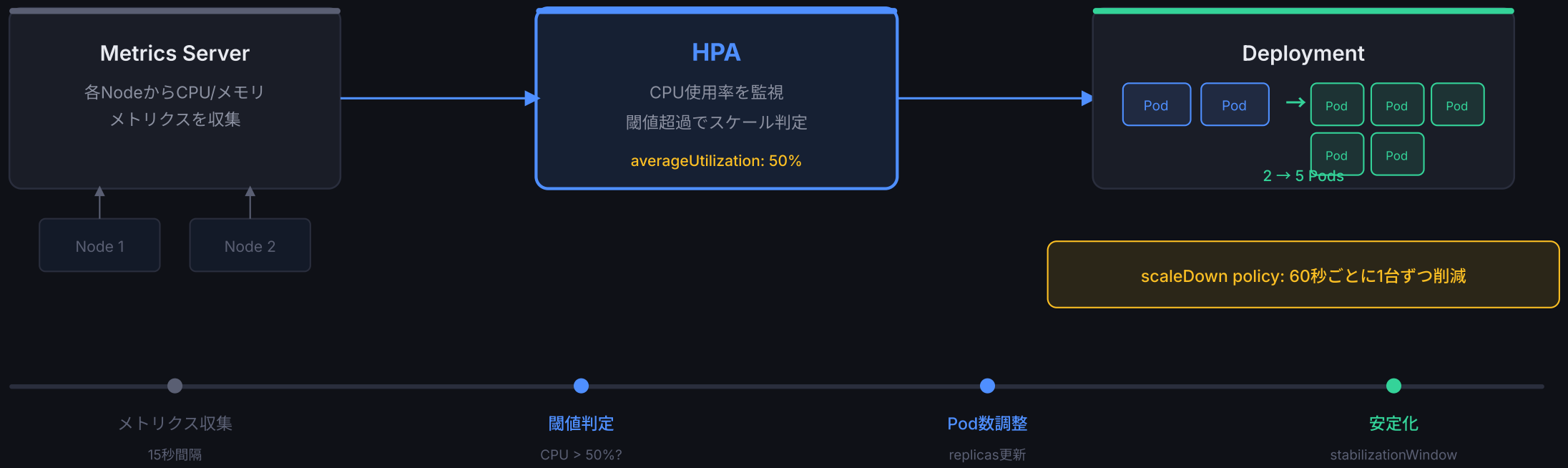
```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-deployment
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

scaleTargetRef
スケール対象のDeploymentを指定する

min/maxReplicas
最小2台〜最大5台の範囲でスケーリング

averageUtilization
requestsに対するCPU使用率50%が閾値

Metrics Server → HPA → Deployment のスケーリングフロー



HPA設計の3つのポイント

01

requestsを正確に設定する

HPAはrequestsに対する使用率で判断する。requestsが不正確だとスケーリングも不正確になる

02

Metrics Serverを事前に導入する

HPAはMetrics Serverのデータがないと CPU使用率を取得できずスケーリングが動かない

03

スケールダウンポリシーを調整する

デフォルトでは急激な縮小でサービス影響が出る。stabilizationWindowで段階的に縮小する